

- Now open the project in your favourite editor (I have used Visual Studio Code with the NativeScript plugin installed).
- NativeScript allows you to install Node modules in the project (although not all of them available out there will work). In this example, we will use the *nativescript-xml2js* plugin for parsing the RSS feed xml. You can install it using the following command:

```
npm install nativescript-xml2js --save
```

- NativeScript uses the MVVM pattern for the separation of concerns. The view is what gets rendered as the UI of the application, and NativeScript has its own XML syntax for it (we will also look at how you can generate one, using code). The model is where the data is stored and the view model file is used for gluing it together. NativeScript supports two-way data binding between the view and the model. Open the *main-view.xml* file and paste the following code:

```
<?xml version="1.0" encoding="UTF-8"?>
<Page xmlns="http://www.nativescript.org/tns.xsd"
loaded="pageLoaded" class="page">
  <Page.actionBar>
    <ActionBar title="RSS Reader Home" />
  </Page.actionBar>
  <GridLayout columns="*" rows="*">
    <ListView row="1" col="0" items="{{ feeds }}">
      <ListView.itemTemplate>
        <GridLayout class="listItem" columns="*"
rows="auto,auto,auto" verticalAlignment="top">
          <Label class="listItemHeader" text="{{ title[0] }}"
textWrap="true" row="0" col="0" verticalAlignment="center" />
          <WebView isUserInteractionEnabled="false" row="1"
col="0" src="{{ description[0] }}" />
          <Label class="listItemReadMore" url="{{ link[0] }}"
text="Read More..." tap="{{ $parents['ListView'].onItemTap
}}" row="2" col="2" />
        </GridLayout>
      </ListView.itemTemplate>
    </ListView>
  </GridLayout>
</Page>
```

Here we have defined a very simple UI. What is inside the `{{}}` uses the data-binding capabilities of NativeScript and the named properties defined are *ViewModel*.

Now open the *main-view-model.js* file and enter the following code:

```
var observableModule = require("data/observable");
var frame = require("ui/frame");
var webViewModule = require("ui/web-view");
var xmlModule = require("tns-core-modules/xml");
var actionBarModule = require("ui/action-bar");
```

```
var pageModule = require("ui/page");
var stackLayoutModule = require("ui/layouts/stack-layout");
var rssFeedLoader = require("./rss-services");
```

```
function MainViewModel() {
  var viewModel = observableModule.fromObject({
    feeds: [],
    onItemTap: function(args) {
      let url = args.view.url;
      frame.topmost().navigate({
        context: {
          url: url
        },
        create: generateWebRenderingPage,
        transition: {
          name: "slide",
          duration: 250,
          curve: "easeIn"
        }
      });
    }
  });
}
```

```
module.exports = MainViewModel;

function generateWebRenderingPage() {
  var webView = new webViewModule.WebView();
  webView.src = this.context.url;

  var stackLayout = new stackLayoutModule.StackLayout();
  stackLayout.addChild(webView);

  var navigationBtn = new actionBarModule.
  NavigationButton();
  navigationBtn.on('tap', function() {
    frame.topmost().goBack();
  });
  navigationBtn.android.systemIcon = "ic_menu_back";
  navigationBtn.text = "back";

  var actionBar = new actionBarModule.ActionBar();
  actionBar.navigationButton = navigationBtn;

  var page = new pageModule.Page();
  page.content = stackLayout;
  page.actionBar = actionBar;

  return page;
}
```